DayProof — Android APK Build Brief for Codex

1. Project Goal

Build a polished Android productivity app called **DayProof**.

The app helps users choose their most important tasks in the morning and honestly review them at night. It should feel beautiful, calm, premium, and emotionally motivating, not like a boring to-do list.

The core loop is simple:

1. User chooses a **morning reminder time**.
2. At that time, the app sends a strong daily reminder.
3. User opens the app and writes the most important tasks for the day.
4. User chooses a **night review time**.
5. At night, the app reminds the user to review the day.
6. User marks each task as:
7. Done
8. Not done
9. Remove
10. Tasks marked **not done** automatically carry over to the next morning.
11. Tasks can be removed manually if the user no longer wants them.

The final deliverable must be an installable Android `.apk`.

---

2. Recommended Stack

Use **Flutter** for the app.

Reason:

- Flutter is ideal for beautiful UI.
- It can generate Android APKs.
- It works well for local-first apps.
- It allows fast UI iteration.
- It is suitable for this type of lightweight productivity app.

Use local storage only. No backend, no login, no Firebase.

Suggested packages:

```
dependencies:
  flutter:
    sdk: flutter

  hive: ^2.2.3
  hive_flutter: ^1.1.0
  flutter_local_notifications: ^17.2.4
  timezone: ^0.9.4
  permission_handler: ^11.3.1
  intl: ^0.19.0
  uuid: ^4.5.1
  google_fonts: ^6.2.1
  flutter_animate: ^4.5.0
  confetti: ^0.7.0
```

If a newer stable version is available, use the newer compatible version.

---

3. Important Android Reminder Behavior

Do not design this as a normal app that force-opens itself from the background without user interaction.

Modern Android restricts automatic background launches. The correct behavior is:

Default Mode

Use scheduled local notifications:

- Morning notification opens the morning planning screen.
- Night notification opens the night review screen.
- The app should open when the user taps the notification.

Optional Alarm Mode

Add an optional setting called:

> Strong reminder mode

When enabled, the app may request exact alarm / alarm-style permissions if needed. This should be handled carefully and explained to the user in simple language.

Use this copy:

> DayProof needs reminder permission so it can remind you exactly when you choose. Without it, reminders may arrive a little late depending on your phone settings.

Do not make the app crash if permission is not granted. Fall back to normal scheduled notifications.

---

4. App Name and Identity

App name: **DayProof**

Tagline:

> Prove your day before it disappears.

Tone:

- Calm
- Honest
- Elegant
- Motivational without being cheesy
- Serious but not stressful

The app should feel like a premium daily ritual.

---

5. Main User Flow

First Launch

Show onboarding screens.

Screen 1:

Title:

> Your day needs proof.

Body:

> Every morning, choose what truly matters. Every night, check whether you protected your day.

Button:

> Start

Screen 2:

Title:

> Pick your morning time.

User selects a time.

Default: 8:00 AM

Screen 3:

Title:

> Pick your night review time.

User selects a time.

Default: 10:00 PM

Screen 4:

Title:

> Keep it small. Keep it real.

Body:

> DayProof works best with 3 to 5 important tasks. Not everything belongs here. Only what matters.

Button:

> Enter DayProof

After onboarding, go to the home screen.

---

6. Main Screens

Build these screens:

1. Onboarding screen
2. Home screen
3. Morning planning screen
4. Night review screen
5. Task history screen

6. Settings screen
7. Stats screen

Use clean navigation.

---

7. Home Screen

The home screen should show the current day status.

Example layout:

```
Good morning, Hasnain

Today needs proof.

3 tasks planned
1 carried over from yesterday

[Plan Today]

Tonight’s review: 10:00 PM
Morning reminder: 8:00 AM
```

If tasks are already planned:

```
Today’s proof

□ Finish experiment notes
□ Call supervisor
□ Clean data table

Night review at 10:00 PM
```

If night review is completed:

```
Day closed.

2 done
1 carried forward

Completion: 67%
```

Bottom navigation:

- Today
- Stats
- History
- Settings

---

8. Morning Planning Screen

Purpose:

Let user add important tasks for the day.

Rules:

- Minimum: 1 task
- Recommended max: 5 tasks
- Hard max: 7 tasks
- Show gentle warning after 5 tasks:

  This app works better when the list is small. Add only what truly matters.

Input field placeholder:

> What must be done today?

Button:

> Add task

Primary button:

> Lock today’s proof

When user taps “Lock today’s proof”, save the tasks for today.

Carry-over tasks should appear at the top in a separate section:

```
Still waiting from yesterday

□ Send email to professor
□ Finish graph labels
```

Each carry-over task should have:

- Keep
- Remove

If user keeps it, it becomes part of today's tasks. If user removes it, it is archived as removed.

---

9. Night Review Screen

Purpose:

Let user honestly review each task.

Title:

> Did you prove your day?

Subtitle:

> No guilt. Just truth.

Each task card should show:

```
Finish experiment notes

[✓ Done] [× Not Done] [Remove]
```

Behavior:

- Done = task completed
- Not Done = task failed and carried over tomorrow
- Remove = task removed and not carried over

When all tasks are reviewed, show a closing screen:

If high completion:

> Strong day. You kept your word.

If medium completion:

> Not perfect, but not wasted.

If low completion:

> Tomorrow starts with the truth.

Show:

```
Today’s score: 3/5
Completion: 60%
Carried to tomorrow: 2
```

Button:

> Close the day

---

10. Carry-Over Logic

Every task should have one of these statuses:

- pending
- done
- failed
- removed

At night:

- Done tasks stay in history.
- Failed tasks move to next morning.
- Removed tasks do not carry over.
- Unreviewed tasks should be treated as pending until the user reviews them.

Next morning:

Show failed tasks from the previous day in the “Still waiting from yesterday” section.

If a task keeps failing across days, track its carry count.

Example:

```
Finish thesis figure
Carried over 3 times
```

After 3 carry-overs, show a gentle message:

> This task has survived 3 days. Break it smaller or remove it.

Provide buttons:

- Break into smaller task
- Keep
- Remove

---

11. Data Model

Create local models.

Task Model

```
class Task {
  final String id;
  String title;
  DateTime createdAt;
  DateTime? completedAt;
  DateTime? reviewedAt;
  String status; // pending, done, failed, removed
  int carryCount;
  DateTime assignedDate;
  DateTime? originalDate;
}
```

DailyProof Model

```
class DailyProof {
  final String id;
  DateTime date;
  List<String> taskIds;
  bool morningLocked;
  bool nightReviewed;
  DateTime? morningCompletedAt;
  DateTime? nightCompletedAt;
  int doneCount;
  int failedCount;
  int removedCount;
}
```

Settings Model

```
class AppSettings {
  TimeOfDay morningTime;
```

```
  TimeOfDay nightTime;
  bool notificationsEnabled;
  bool strongReminderMode;
  bool onboardingCompleted;
  int maxTasks;
}
```

Use Hive or another simple local database.

---

12. Notification Requirements

Implement daily notifications:

Morning Notification

Title:

> DayProof

Body:

> Choose what matters today.

On tap:

Open morning planning screen.

Night Notification

Title:

> DayProof Review

Body:

> Did you do what you promised yourself?

On tap:

Open night review screen.

Missed Review Notification

Optional:

If the user ignores the night review, show a softer reminder 30 minutes later:

> Still want to close the day?

Do not annoy the user repeatedly.

---

13. Notification Permission Flow

On Android 13 and above, request notification permission.

If permission is denied, show non-blocking message:

> Reminders are off. You can still use DayProof manually.

In settings, provide:

- Enable notifications
- Change morning time
- Change night review time
- Strong reminder mode
- Reset onboarding
- Export data as JSON
- Clear all data

---

14. UI Direction

The UI should be beautiful enough to feel like a serious personal app.

Design style:

- Dark premium interface
- Soft gradients
- Glass-like cards
- Smooth animations
- Rounded corners
- Excellent typography
- Calm spacing
- Subtle glowing accents
- No childish colors
- No clutter

Suggested color palette:

```
Background: #09090B
Card: #141418
Card border: rgba(255,255,255,0.08)
Primary accent: #A78BFA
Secondary accent: #22D3EE
Success: #34D399
Failure: #FB7185
Text primary: #F8FAFC
Text secondary: #A1A1AA
```

Use the color palette in code.

Typography:

Use Google Font:

- Inter
- Plus Jakarta Sans
- Manrope

Preferred: **Plus Jakarta Sans**

---

15. Visual Details

Use animations, but keep them smooth and light.

Examples:

- Task cards slide in softly.
- Done button gives a small satisfying animation.
- Failed task shakes subtly once.
- Completion screen fades in.
- Confetti appears only when all tasks are done.
- Morning screen has a subtle sunrise gradient.
- Night screen has a calm moon/dark gradient.

Do not overdo animations.

---

16. App Personality

Use short, human copy.

Good examples:

```
Today needs proof.
Choose only what matters.
No guilt. Just truth.
Close the day honestly.
This task keeps coming back.
Make tomorrow easier.
```

Avoid generic motivational quotes like:

```
You can do anything!
Believe in yourself!
Crush your goals!
```

The app should feel mature.

---

17. Task Rules

Implement these exact rules:

1. User can add tasks in the morning.
2. User can edit task text before locking the day.
3. User can delete task before locking the day.
4. After locking, user can still add emergency tasks, but they should be marked as added later.
5. At night, user reviews all active tasks.
6. Done tasks are saved as done.
7. Failed tasks are carried to next morning.
8. Removed tasks are archived and do not return.
9. If a task fails 3 days in a row, prompt the user to break it down.
10. The app must not delete data silently.

---

18. Stats Screen

Show simple statistics.

Cards:

```
This week
Completion: 72%

Current streak
4 days reviewed
```

```
Tasks completed
23

Tasks carried
6
```

Add a weekly bar chart.

No need for complex analytics.

Stats should include:

- Daily completion percentage
- Weekly completion percentage
- Total tasks completed
- Total failed tasks
- Total removed tasks
- Review streak
- Best day
- Most carried task

---

19. History Screen

Show previous days.

Example:

```
Monday, June 22
3/5 completed
2 carried forward

Tuesday, June 23
5/5 completed
Perfect day
```

When user taps a day, show tasks:

```
✓ Finish notes
✓ Clean graph
× Send email
Removed: Buy notebook
```

20. Settings Screen

Settings list:

```
Morning reminder time
Night review time
Notifications
Strong reminder mode
Max tasks per day
Export data
Clear all data
About DayProof
```

Changing reminder times must reschedule notifications.

For clear all data:

Ask confirmation:

> This will erase all tasks, history, and stats. This cannot be undone.

Buttons:

- Cancel
- Clear everything

---

21. Empty States

If no tasks:

```
Nothing planned yet.

Choose the few things that would make today count.
```

If no history:

```
No proof yet.

Finish one day and it will appear here.
```

If notifications disabled:

```
Reminders are off.

DayProof works best when it can call you back to your day.
```

---

22. Error Handling

Handle these cases gracefully:

- Notification permission denied
- Exact alarm permission unavailable
- Local database error
- Empty task text
- Too many tasks
- User changes phone date/time
- User misses night review
- User opens morning screen after noon
- User opens night review before tasks are planned

Never crash.

---

23. Folder Structure

Use a clean Flutter structure:

```
lib/
  main.dart
  app.dart

  core/
    theme/
      app_theme.dart
      app_colors.dart
      app_text_styles.dart
    utils/
      date_utils.dart
      task_utils.dart

  data/
    models/
      task_model.dart
```

```
    daily_proof_model.dart
    app_settings_model.dart
  storage/
    local_storage_service.dart

services/
  notification_service.dart
  reminder_scheduler.dart
  permission_service.dart

features/
  onboarding/
    onboarding_screen.dart
  today/
    home_screen.dart
    morning_planning_screen.dart
    night_review_screen.dart
    widgets/
      task_card.dart
      carry_over_card.dart
      proof_summary_card.dart
  stats/
    stats_screen.dart
    widgets/
      weekly_chart.dart
  history/
    history_screen.dart
    day_detail_screen.dart
  settings/
    settings_screen.dart

shared/
  widgets/
    glass_card.dart
    primary_button.dart
    empty_state.dart
    time_picker_tile.dart
```

---

24. APK Build Requirement

After implementation, build the APK.

Run:

```
flutter clean
flutter pub get
flutter analyze
flutter test
flutter build apk --release
```

The APK should be generated at:

```
build/app/outputs/flutter-apk/app-release.apk
```

Also create a copy at project root:

```
release/dayproof-release.apk
```

If release signing is not configured, generate a debug APK too:

```
flutter build apk --debug
```

Copy it to:

```
release/dayproof-debug.apk
```

Make sure the final answer from Codex clearly tells me where the APK is located.

---

25. App Icon and Splash

Create a simple app icon.

Icon concept:

- Dark background
- Minimal check mark
- Small sunrise/moon split
- Premium look

Use a clean adaptive Android icon.

Add splash screen:

Text:

DayProof

Subtitle:

Prove your day.

---

26. README Requirement

Create a beautiful `README.md`.

The README should include:

- App name
- Short description
- Screenshots placeholder section
- Features
- Tech stack
- How to run
- How to build APK
- Folder structure
- Notes about Android reminder permissions
- License

The README must feel natural and human-written. Avoid robotic, generic wording.

Do not write anything like:

This project was generated by AI.

Do not write:

As an AI language model.

Do not use fake screenshots. Add screenshot placeholders only if actual screenshots are not available.

---

27. Commit Style

Use natural commit messages.

Examples:

```
Set up Flutter project structure
Add local task storage
```

```
Build morning planning flow
Add night review and carry-over logic
Wire daily reminders
Polish DayProof visual design
Add APK build instructions
```

Avoid robotic commits like:

```
feat: implement comprehensive user interface functionality
chore: update files
AI generated app
```

---

28. Acceptance Criteria

The app is complete only when:

- It runs on Android.
- User can complete onboarding.
- User can set morning and night times.
- Morning notification is scheduled.
- Night notification is scheduled.
- User can add tasks.
- User can lock today's tasks.
- User can review tasks at night.
- Done tasks are saved.
- Failed tasks carry over to the next morning.
- Removed tasks do not return.
- Stats screen works.
- History screen works.
- Settings screen works.
- App survives restart.
- Data persists locally.
- Notifications can be rescheduled.
- APK builds successfully.
- APK is copied to the `release/` folder.

---

29. Testing Checklist

Manually test:

First Launch

- App opens onboarding.
- User can set morning time.
- User can set night time.
- App goes to home screen.

Morning Flow

- Add one task.
- Add five tasks.
- Try adding empty task.
- Try exceeding max tasks.
- Lock the day.
- Restart app and confirm tasks remain.

Night Flow

- Mark task done.
- Mark task failed.
- Remove task.
- Close day.
- Confirm score is correct.

Carry-Over

- Fail a task.
- Move to next day using test mode or date simulation.
- Confirm failed task appears next morning.
- Remove carried task.
- Confirm removed task does not return.

Settings

- Change morning time.
- Change night time.
- Confirm reminders reschedule.
- Disable notifications.
- Re-enable notifications.
- Export data.
- Clear all data.

APK

- Build release APK.
- Install APK on Android phone.
- Confirm app opens.
- Confirm notifications work after granting permission.

---

30. Developer Test Mode

Add a hidden developer test mode to make testing easier.

In Settings, if user taps the app version 7 times, unlock:

```
Developer Test Mode
```

Developer tools:

- Trigger morning notification now
- Trigger night notification now
- Simulate next day
- Reset today only
- Print local database summary

This is only for testing and should not disturb normal users.

---

31. Final Codex Instructions

Implement the full Flutter app.

Focus on:

1. Reliable local storage
2. Correct carry-over logic
3. Beautiful UI
4. Working scheduled notifications
5. Successful APK generation

Do not overcomplicate the app.

Do not add login.

Do not add cloud sync.

Do not add ads.

Do not add social features.

Do not add AI features.

This should be a clean, elegant, local-first Android app that does one thing extremely well:

It asks the user what matters in the morning and asks for proof at night.

When done, provide:

1. Summary of what was built
2. Commands used
3. APK path
4. Any limitations
5. Next improvements